

Limited-Memory BFGS (L-BFGS)

Quasi-Newton Methods for Large-Scale Optimization

Robert Hildebrand

Industrial and Systems Engineering
Virginia Tech

Reading: Nocedal & Wright, Chapter 7

Recap: The BFGS Inverse Hessian Update

BFGS maintains $\mathbf{H}_k \approx [\nabla^2 f(\mathbf{x}_k)]^{-1}$ via the rank-2 update:

$$\mathbf{H}_{k+1} = (\mathbf{I} - \rho_k \mathbf{s}_k \mathbf{y}_k^\top) \mathbf{H}_k (\mathbf{I} - \rho_k \mathbf{y}_k \mathbf{s}_k^\top) + \rho_k \mathbf{s}_k \mathbf{s}_k^\top$$

where $\mathbf{s}_k = \mathbf{x}_{k+1} - \mathbf{x}_k$, $\mathbf{y}_k = \nabla f_{k+1} - \nabla f_k$, $\rho_k = 1/(\mathbf{y}_k^\top \mathbf{s}_k)$.

The problem for large n :

- $\mathbf{H}_k$ is a dense $n \times n$ matrix $\Rightarrow O(n^2)$ **storage**
- Matrix-vector product $\mathbf{H}_k \nabla f_k$ costs $O(n^2)$
- For $n = 10^6$ (common in ML), storing $\mathbf{H}_k$ requires ~ 8 TB

Can we get the benefits of BFGS without storing $\mathbf{H}_k$ explicitly?

The Key Observation

The BFGS update is **recursive**: $\mathbf{H}_{k+1}$ is defined entirely in terms of $\mathbf{H}_k$, $\mathbf{s}_k$, and $\mathbf{y}_k$.

Unrolling m steps backward from $\mathbf{H}_k$:

$$\mathbf{H}_k = f(\mathbf{H}_{k-m}, \mathbf{s}_{k-m}, \mathbf{y}_{k-m}, \mathbf{s}_{k-m+1}, \mathbf{y}_{k-m+1}, \dots, \mathbf{s}_{k-1}, \mathbf{y}_{k-1})$$

L-BFGS idea: Instead of storing the dense matrix $\mathbf{H}_k$, store only:

- The last m pairs $\{\mathbf{s}_i, \mathbf{y}_i\}$ for $i = k - m, \dots, k - 1$
- A simple initial matrix $\mathbf{H}_k^0$ (typically a scalar times $\mathbf{I}$)

Then **implicitly** compute $\mathbf{H}_k \nabla f_k$ by applying the m BFGS updates to $\mathbf{H}_k^0 \nabla f_k$ one at a time.

	BFGS	L-BFGS
Storage	$O(n^2)$	$O(mn)$
Cost of $\mathbf{H}_k \nabla f_k$	$O(n^2)$	$O(mn)$

Unrolling the BFGS Recursion

Define the shorthand $V_i = \mathbf{I} - \rho_i \mathbf{y}_i \mathbf{s}_i^\top$. Then the BFGS update is:

$$\mathbf{H}_{k+1} = V_k^\top \mathbf{H}_k V_k + \rho_k \mathbf{s}_k \mathbf{s}_k^\top$$

Unrolling m steps starting from initial matrix $\mathbf{H}_k^0$:

$$\begin{aligned}\mathbf{H}_k &= V_{k-1}^\top \mathbf{H}_{k-1} V_{k-1} + \rho_{k-1} \mathbf{s}_{k-1} \mathbf{s}_{k-1}^\top \\ &= V_{k-1}^\top (V_{k-2}^\top \mathbf{H}_{k-2} V_{k-2} + \rho_{k-2} \mathbf{s}_{k-2} \mathbf{s}_{k-2}^\top) V_{k-1} + \rho_{k-1} \mathbf{s}_{k-1} \mathbf{s}_{k-1}^\top \\ &= \dots\end{aligned}$$

After m steps:

$$\begin{aligned}\mathbf{H}_k &= (V_{k-1}^\top \cdots V_{k-m}^\top) \mathbf{H}_k^0 (V_{k-m} \cdots V_{k-1}) \\ &\quad + \rho_{k-m} (V_{k-1}^\top \cdots V_{k-m+1}^\top) \mathbf{s}_{k-m} \mathbf{s}_{k-m}^\top (V_{k-m+1} \cdots V_{k-1}) \\ &\quad + \dots + \rho_{k-1} \mathbf{s}_{k-1} \mathbf{s}_{k-1}^\top\end{aligned}$$

We never form this matrix. We only need to compute $\mathbf{H}_k \nabla f_k$.

Computing $\mathbf{H}_k \nabla f_k$: The Idea

To compute $\mathbf{r} = \mathbf{H}_k \nabla f_k$ from the unrolled formula, note the structure:

$$\mathbf{H}_k \nabla f_k = (V_{k-1}^\top \cdots V_{k-m}^\top) \mathbf{H}_k^0 (V_{k-m} \cdots V_{k-1}) \nabla f_k + \text{correction terms}$$

Each $V_i \mathbf{q} = (\mathbf{I} - \rho_i \mathbf{y}_i \mathbf{s}_i^\top) \mathbf{q} = \mathbf{q} - \rho_i (\mathbf{s}_i^\top \mathbf{q}) \mathbf{y}_i$ is just:

- One inner product $\mathbf{s}_i^\top \mathbf{q}$: $O(n)$
- One “axpy” $\mathbf{q} - \alpha \mathbf{y}_i$: $O(n)$

Similarly $V_i^\top \mathbf{q} = \mathbf{q} - \rho_i (\mathbf{y}_i^\top \mathbf{q}) \mathbf{s}_i$ costs $O(n)$.

Strategy:

- 1 **Right-multiply** ∇f_k by $V_{k-1}, V_{k-2}, \dots, V_{k-m}$ (peeling off layers)
- 2 **Multiply** by $\mathbf{H}_k^0$ (cheap if $\mathbf{H}_k^0 = \gamma_k \mathbf{I}$)
- 3 **Left-multiply** by $V_{k-m}^\top, V_{k-m+1}^\top, \dots, V_{k-1}^\top$ (adding layers back)

Total cost: $O(mn)$ — **linear in n** for fixed m .

The Two-Loop Recursion

Algorithm (Nocedal & Wright, Alg. 7.4):

Loop 1 (right to left):

$$\mathbf{q} \leftarrow \nabla f_k$$

for $i = k - 1, k - 2, \dots, k - m$:

$$\alpha_i \leftarrow \rho_i \mathbf{s}_i^\top \mathbf{q}$$

$$\mathbf{q} \leftarrow \mathbf{q} - \alpha_i \mathbf{y}_i$$

$$\mathbf{r} \leftarrow \mathbf{H}_k^0 \mathbf{q}$$

Loop 2 (left to right):

for $i = k - m, k - m + 1, \dots, k - 1$:

$$\beta \leftarrow \rho_i \mathbf{y}_i^\top \mathbf{r}$$

$$\mathbf{r} \leftarrow \mathbf{r} + (\alpha_i - \beta) \mathbf{s}_i$$

return $\mathbf{r} = \mathbf{H}_k \nabla f_k$

Cost analysis:

Each loop iteration:

- 1 inner product: $O(n)$
- 1 vector update: $O(n)$

m iterations per loop.

Total: $4mn$ multiplications

Storage: $2mn + n$ floats
(the m pairs $\{\mathbf{s}_i, \mathbf{y}_i\}$ plus $\mathbf{q}$)

Typical: $m = 3\text{--}20$

Why Does the Two-Loop Recursion Work?

Loop 1 computes $\mathbf{q} = V_{k-m} \cdots V_{k-1} \nabla f_k$:

Starting with $\mathbf{q} = \nabla f_k$, each step applies

$$\mathbf{q} \leftarrow V_i \mathbf{q} = \mathbf{q} - \rho_i (\mathbf{s}_i^\top \mathbf{q}) \mathbf{y}_i$$

and stores $\alpha_i = \rho_i \mathbf{s}_i^\top \mathbf{q}$ (the coefficient *before* applying V_i).

Middle step: $\mathbf{r} = \mathbf{H}_k^0 \mathbf{q}$ applies the initial Hessian approximation.

Loop 2 applies the left-multiplications $V_{k-m}^\top, \dots, V_{k-1}^\top$ and adds the rank-1 correction terms $\rho_i \mathbf{s}_i \mathbf{s}_i^\top$:

$$\mathbf{r} \leftarrow V_i^\top \mathbf{r} + \rho_i \mathbf{s}_i (\mathbf{s}_i^\top V_{i+1} \cdots V_{k-1} \nabla f_k)$$

The stored α_i values carry exactly the information needed from Loop 1.

The result is $\mathbf{r} = \mathbf{H}_k \nabla f_k$ as if we had formed the full matrix — but using only $O(mn)$ work.

Example: Memory Buffer with $m = 2$

At each iteration, L-BFGS stores at most $m = 2$ pairs $\{\mathbf{s}_i, \mathbf{y}_i\}$.

Iteration k	Stored pairs	Action
$k = 0$	$\{\}$ (empty)	Steepest descent: $\mathbf{p}_0 = -\nabla f_0$ Compute $\mathbf{s}_0, \mathbf{y}_0$ after line search
$k = 1$	$\{(\mathbf{s}_0, \mathbf{y}_0)\}$	1 pair $\Rightarrow$ 1-step recursion Compute $\mathbf{s}_1, \mathbf{y}_1$
$k = 2$	$\{(\mathbf{s}_0, \mathbf{y}_0), (\mathbf{s}_1, \mathbf{y}_1)\}$	Buffer full (2 pairs) Compute $\mathbf{s}_2, \mathbf{y}_2$
$k = 3$	$\{(\mathbf{s}_1, \mathbf{y}_1), (\mathbf{s}_2, \mathbf{y}_2)\}$	Discard $(\mathbf{s}_0, \mathbf{y}_0)$ Compute $\mathbf{s}_3, \mathbf{y}_3$
$k = 4$	$\{(\mathbf{s}_2, \mathbf{y}_2), (\mathbf{s}_3, \mathbf{y}_3)\}$	Discard $(\mathbf{s}_1, \mathbf{y}_1)$

Numerical Example: Setup

Consider minimizing $f(\mathbf{x}) = x_1^2 + 2x_2^2$ (diagonal Hessian $\nabla^2 f = \text{diag}(2, 4)$).

Suppose at iteration $k = 4$ with $m = 2$, the buffer contains:

$$\mathbf{s}_2 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad \mathbf{y}_2 = \begin{pmatrix} 2 \\ 0 \end{pmatrix}, \quad \rho_2 = \frac{1}{\mathbf{y}_2^\top \mathbf{s}_2} = \frac{1}{2} \quad \mathbf{s}_3 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \quad \mathbf{y}_3 = \begin{pmatrix} 0 \\ 4 \end{pmatrix}, \quad \rho_3 = \frac{1}{\mathbf{y}_3^\top \mathbf{s}_3} = \frac{1}{4}$$

Current gradient: $\nabla f_4 = \begin{pmatrix} 6 \\ 8 \end{pmatrix}$

Initial scaling: $\gamma_4 = \frac{\mathbf{s}_3^\top \mathbf{y}_3}{\mathbf{y}_3^\top \mathbf{y}_3} = \frac{4}{16} = \frac{1}{4}$, so $\mathbf{H}_4^0 = \frac{1}{4} \mathbf{I}$

Goal: Compute $\mathbf{r} = \mathbf{H}_4 \nabla f_4$ using the two-loop recursion.

Numerical Example: Loop 1 (Right to Left)

Initialize: $\mathbf{q} = \nabla f_4 = \begin{pmatrix} 6 \\ 8 \end{pmatrix}$

$i = 3$ (**most recent pair**):

$$\alpha_3 = \rho_3 \mathbf{s}_3^\top \mathbf{q} = \frac{1}{4} \begin{pmatrix} 0 \\ 1 \end{pmatrix}^\top \begin{pmatrix} 6 \\ 8 \end{pmatrix} = \frac{1}{4}(8) = 2 \quad (\text{store } \alpha_3 = 2)$$

$$\mathbf{q} \leftarrow \mathbf{q} - \alpha_3 \mathbf{y}_3 = \begin{pmatrix} 6 \\ 8 \end{pmatrix} - 2 \begin{pmatrix} 0 \\ 4 \end{pmatrix} = \begin{pmatrix} 6 \\ 0 \end{pmatrix}$$

$i = 2$ (**older pair**):

$$\alpha_2 = \rho_2 \mathbf{s}_2^\top \mathbf{q} = \frac{1}{2} \begin{pmatrix} 1 \\ 0 \end{pmatrix}^\top \begin{pmatrix} 6 \\ 0 \end{pmatrix} = \frac{1}{2}(6) = 3 \quad (\text{store } \alpha_2 = 3)$$

$$\mathbf{q} \leftarrow \mathbf{q} - \alpha_2 \mathbf{y}_2 = \begin{pmatrix} 6 \\ 0 \end{pmatrix} - 3 \begin{pmatrix} 2 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

Numerical Example: Middle Step and Loop 2

Middle step: $\mathbf{r} = \mathbf{H}_4^0 \mathbf{q} = \frac{1}{4} \begin{pmatrix} 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$

Loop 2 (left to right):

$i = 2$:

$$\beta = \rho_2 \mathbf{y}_2^\top \mathbf{r} = \frac{1}{2} \begin{pmatrix} 2 \\ 0 \end{pmatrix}^\top \begin{pmatrix} 0 \\ 0 \end{pmatrix} = 0$$

$$\mathbf{r} \leftarrow \mathbf{r} + (\alpha_2 - \beta) \mathbf{s}_2 = \begin{pmatrix} 0 \\ 0 \end{pmatrix} + (3 - 0) \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 3 \\ 0 \end{pmatrix}$$

$i = 3$:

$$\beta = \rho_3 \mathbf{y}_3^\top \mathbf{r} = \frac{1}{4} \begin{pmatrix} 0 \\ 4 \end{pmatrix}^\top \begin{pmatrix} 3 \\ 0 \end{pmatrix} = 0$$

$$\mathbf{r} \leftarrow \mathbf{r} + (\alpha_3 - \beta) \mathbf{s}_3 = \begin{pmatrix} 3 \\ 0 \end{pmatrix} + (2 - 0) \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \boxed{\begin{pmatrix} 3 \\ 2 \end{pmatrix}}$$

Numerical Example: Verification

The two-loop recursion returned $\mathbf{H}_4 \nabla f_4 = \begin{pmatrix} 3 \\ 2 \end{pmatrix}$.

Check: The true inverse Hessian is $[\nabla^2 f]^{-1} = \text{diag}(\frac{1}{2}, \frac{1}{4})$, so:

$$[\nabla^2 f]^{-1} \nabla f_4 = \begin{pmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{4} \end{pmatrix} \begin{pmatrix} 6 \\ 8 \end{pmatrix} = \begin{pmatrix} 3 \\ 2 \end{pmatrix} \quad \checkmark$$

The L-BFGS result **matches exactly** because the two stored pairs $(\mathbf{s}_2, \mathbf{y}_2)$ and $(\mathbf{s}_3, \mathbf{y}_3)$ happen to encode the full curvature of this 2D quadratic.

Key point

With $m = n = 2$ pairs, L-BFGS recovers the exact inverse Hessian for a quadratic. In general, $m < n$ gives an *approximation* — but even $m \ll n$ captures the most important curvature directions.

The search direction is $\mathbf{p}_4 = -\mathbf{H}_4 \nabla f_4 = \begin{pmatrix} -3 \\ -2 \end{pmatrix}$, pointing toward the minimizer.

What If $m < n$? Example with $m = 1$

Same problem: $f(\mathbf{x}) = x_1^2 + 2x_2^2$, $\nabla f_4 = (6, 8)^\top$. But now $m = 1$: only the most recent pair.

Buffer: $\mathbf{s}_3 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$, $\mathbf{y}_3 = \begin{pmatrix} 0 \\ 4 \end{pmatrix}$, $\rho_3 = \frac{1}{4}$. $\gamma_4 = \frac{\mathbf{s}_3^\top \mathbf{y}_3}{\mathbf{y}_3^\top \mathbf{y}_3} = \frac{4}{16} = \frac{1}{4}$, $\mathbf{H}_4^0 = \frac{1}{4}\mathbf{I}$.

Loop 1 ($i = 3$ only): $\alpha_3 = \rho_3 \mathbf{s}_3^\top \nabla f_4 = \frac{1}{4}(8) = 2$, $\mathbf{q} = \begin{pmatrix} 6 \\ 8 \end{pmatrix} - 2 \begin{pmatrix} 0 \\ 4 \end{pmatrix} = \begin{pmatrix} 6 \\ 0 \end{pmatrix}$

Middle: $\mathbf{r} = \frac{1}{4} \begin{pmatrix} 6 \\ 0 \end{pmatrix} = \begin{pmatrix} 1.5 \\ 0 \end{pmatrix}$

Loop 2 ($i = 3$ only): $\beta = \rho_3 \mathbf{y}_3^\top \mathbf{r} = \frac{1}{4}(0) = 0$, $\mathbf{r} = \begin{pmatrix} 1.5 \\ 0 \end{pmatrix} + (2 - 0) \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \boxed{\begin{pmatrix} 1.5 \\ 2 \end{pmatrix}}$

Compare: True $[\nabla^2 f]^{-1} \nabla f_4 = (3, 2)^\top$, but L-BFGS gives $(1.5, 2)^\top$.

The x_2 -curvature is captured exactly (from the stored pair), but the x_1 -curvature uses only the crude initial scaling $\gamma_4 = \frac{1}{4}$ instead of the true $\frac{1}{2}$.

Choosing the Initial Matrix $\mathbf{H}_k^0$

The simplest choice: $\mathbf{H}_k^0 = \gamma_k \mathbf{I}$ where

$$\gamma_k = \frac{\mathbf{s}_{k-1}^\top \mathbf{y}_{k-1}}{\mathbf{y}_{k-1}^\top \mathbf{y}_{k-1}}$$

Why this scaling?

- Along the most recent step direction, $\gamma_k \approx \mathbf{s}_{k-1}^\top \mathbf{s}_{k-1} / (\mathbf{s}_{k-1}^\top \mathbf{y}_{k-1})$, which is a scalar approximation to the inverse Hessian eigenvalue along $\mathbf{s}_{k-1}$.
- Equivalently: $\gamma_k \mathbf{I}$ satisfies $\gamma_k \mathbf{y}_{k-1} \approx \mathbf{s}_{k-1}$ in the least-squares sense (minimizes $\|\gamma \mathbf{y}_{k-1} - \mathbf{s}_{k-1}\|^2$).
- This ensures the initial scaling is adapted to the **local curvature**, preventing the first few L-BFGS iterations from behaving like steepest descent.

Note: $\mathbf{H}_k^0$ is **recomputed at every iteration** — it is not carried forward. This is one reason L-BFGS is robust: even if old curvature information degrades, the scaling resets.

The Full L-BFGS Algorithm

- 0 Choose $\mathbf{x}_0$, memory size m (e.g., $m = 10$), set $k = 0$
- 1 Compute $\nabla f_k = \nabla f(\mathbf{x}_k)$. If $\|\nabla f_k\| < \varepsilon$, stop.
- 2 Compute search direction $\mathbf{p}_k = -\mathbf{H}_k \nabla f_k$ via the **two-loop recursion** using $\{\mathbf{s}_i, \mathbf{y}_i\}_{i=k-m}^{k-1}$ and $\mathbf{H}_k^0 = \gamma_k \mathbf{I}$
- 3 Line search: find α_k satisfying the **Wolfe conditions**
- 4 Set $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$
- 5 Compute and store $\mathbf{s}_k = \mathbf{x}_{k+1} - \mathbf{x}_k$, $\mathbf{y}_k = \nabla f_{k+1} - \nabla f_k$
If more than m pairs stored, discard the oldest.
- 6 Set $k = k + 1$, go to Step 1

Storage and Cost Summary

	Steepest Descent	BFGS	L-BFGS
Storage	$O(n)$	$O(n^2)$	$O(mn)$
Cost/iteration	$O(n)$	$O(n^2)$	$O(mn)$
Convergence	Linear	Superlinear	Superlinear*
Requires	∇f	∇f	∇f

*L-BFGS convergence is superlinear for sufficiently large m (and under standard assumptions). In practice, even small m gives near-superlinear behavior.

The sweet spot: L-BFGS gives most of BFGS's convergence benefit at a cost that scales like steepest descent. For $m = 10$ and $n = 10^6$:

- BFGS: 8×10^{12} bytes $\approx$ 8 TB
- L-BFGS: $2 \times 10 \times 10^6 \times 8$ bytes = 160 MB

How Large Should m Be?

Empirical findings (Nocedal & Wright; Liu & Nocedal, 1989):

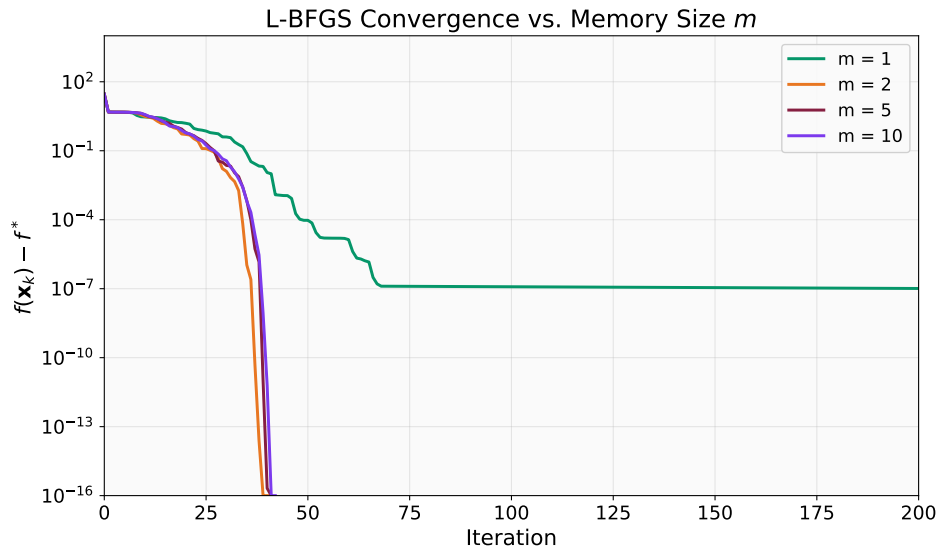
- $m = 3\text{--}5$: Already a large improvement over steepest descent
- $m = 5\text{--}20$: Typical range; diminishing returns beyond $m \approx 20$
- $m = n$: Equivalent to full BFGS (but defeats the purpose)

Why does a small m suffice?

- Near a minimizer, the Hessian changes slowly $\Rightarrow$ old curvature pairs $(\mathbf{s}_i, \mathbf{y}_i)$ still carry useful information even after being “forgotten”
- The initial scaling γ_k adapts to the current curvature at each step, compensating for the limited memory
- The Wolfe line search ensures $\mathbf{y}_k^\top \mathbf{s}_k > 0$, maintaining well-conditioned updates regardless of history length

Rule of thumb: Start with $m = 10$. Increase if convergence is slow; decrease if memory is tight.

Effect of Memory Size on Convergence



What Is Lost with Limited Memory?

Full BFGS accumulates curvature information from *all* previous iterations. After k steps, $\mathbf{H}_k$ reflects k secant conditions.

L-BFGS only reflects the most recent m secant conditions. Information from earlier iterations is discarded.

Consequences:

- Full BFGS: $\mathbf{H}_k \rightarrow [\nabla^2 f(\mathbf{x}^*)]^{-1}$ as $k \rightarrow \infty$ (Dennis–Moré)
L-BFGS: $\mathbf{H}_k$ does *not* converge to the true inverse Hessian in general.
- Full BFGS on a quadratic with exact line search: terminates in $\leq n$ steps.
L-BFGS: may require more than n steps since old conjugacy is forgotten.
- Full BFGS: superlinear convergence guaranteed (under standard assumptions).
L-BFGS: linear convergence provable; superlinear observed empirically for moderate m .

In practice, the loss is small for most problems, and the $O(n^2) \rightarrow O(mn)$ savings make L-BFGS applicable to problems where BFGS is infeasible.

Compact Representation (Optional)

For advanced implementations, $\mathbf{H}_k$ admits a **compact representation**:

$$\mathbf{H}_k = \gamma_k \mathbf{I} + [\gamma_k S_k \quad Y_k] \underbrace{\begin{bmatrix} -\gamma_k S_k^\top Y_k & L_k \\ L_k^\top & D_k \end{bmatrix}^{-1}}_{2m \times 2m} \begin{bmatrix} \gamma_k S_k^\top \\ Y_k^\top \end{bmatrix}$$

where $S_k = [\mathbf{s}_{k-m} \cdots \mathbf{s}_{k-1}]$, $Y_k = [\mathbf{y}_{k-m} \cdots \mathbf{y}_{k-1}]$ are $n \times m$, and L_k, D_k are $m \times m$ matrices extracted from $S_k^\top Y_k$.

Why useful?

- Enables efficient $\mathbf{H}_k \mathbf{v}$ for *any* vector $\mathbf{v}$, not just ∇f_k
- Essential for **L-BFGS-B** (bound-constrained L-BFGS), where the subspace minimization requires multiple products with $\mathbf{H}_k$
- The inner $2m \times 2m$ system is tiny and can be solved directly

Practical Considerations

When to use L-BFGS:

- Large-scale smooth unconstrained (or bound-constrained) optimization
- When ∇f is available but $\nabla^2 f$ is not (or is too expensive)
- $n \gtrsim 1000$ where full BFGS is impractical

When L-BFGS may struggle:

- **Nonsmooth problems:** $\mathbf{y}_k^\top \mathbf{s}_k$ may not be positive; use damping or subgradient methods
- **Stochastic gradients:** Noisy gradient estimates corrupt the curvature pairs $(\mathbf{s}_k, \mathbf{y}_k)$; requires modifications (e.g., mL-BFGS, overlap L-BFGS)
- **Highly nonconvex problems:** The positive-definiteness of $\mathbf{H}_k$ may poorly approximate an indefinite true Hessian

Default optimizer in many codes: SciPy `minimize`, Julia `Optim.jl`, MATLAB `fminunc`, TensorFlow Probability, and most PINN training pipelines.

Summary

- 1 **BFGS** builds a full $n \times n$ inverse Hessian approximation: $O(n^2)$ cost.
- 2 **L-BFGS** stores only m vector pairs $\{\mathbf{s}_i, \mathbf{y}_i\}$: $O(mn)$ cost.
- 3 The **two-loop recursion** computes $\mathbf{H}_k \nabla f_k$ implicitly by applying m rank-1 updates to $\mathbf{H}_k^0 \nabla f_k$ — no matrix ever formed.
- 4 The initial scaling $\gamma_k = \mathbf{s}_{k-1}^\top \mathbf{y}_{k-1} / \|\mathbf{y}_{k-1}\|^2$ adapts to local curvature.
- 5 Typical choice $m = 3\text{--}20$ gives near-BFGS performance at near-gradient-descent cost.
- 6 L-BFGS is the **standard workhorse** for large-scale smooth optimization.